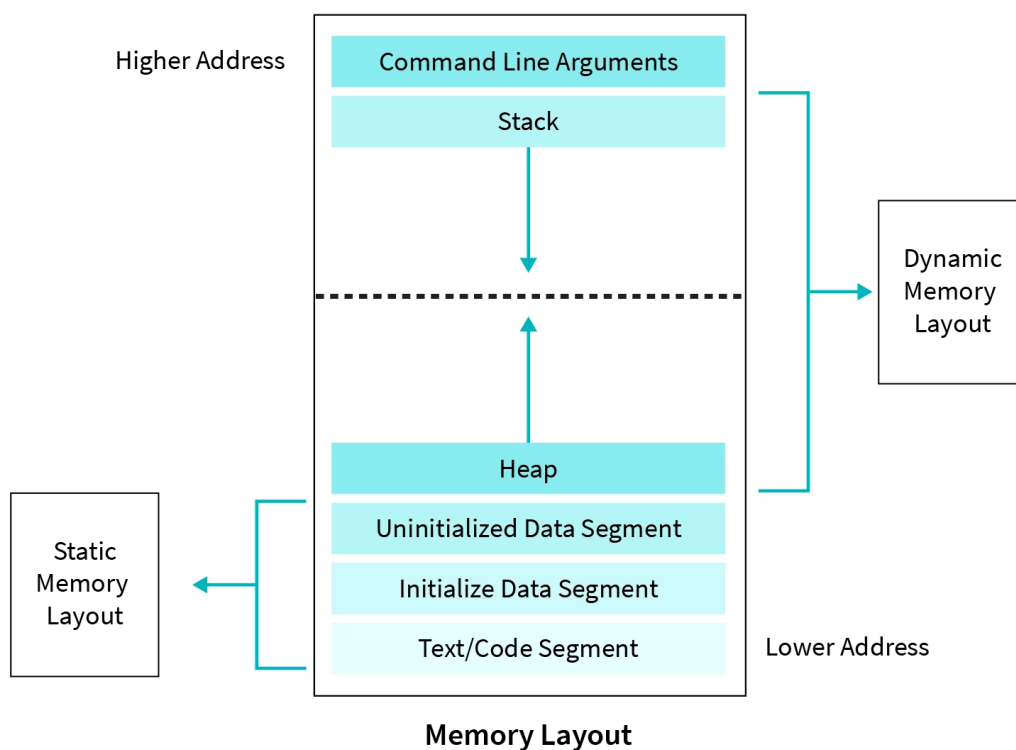


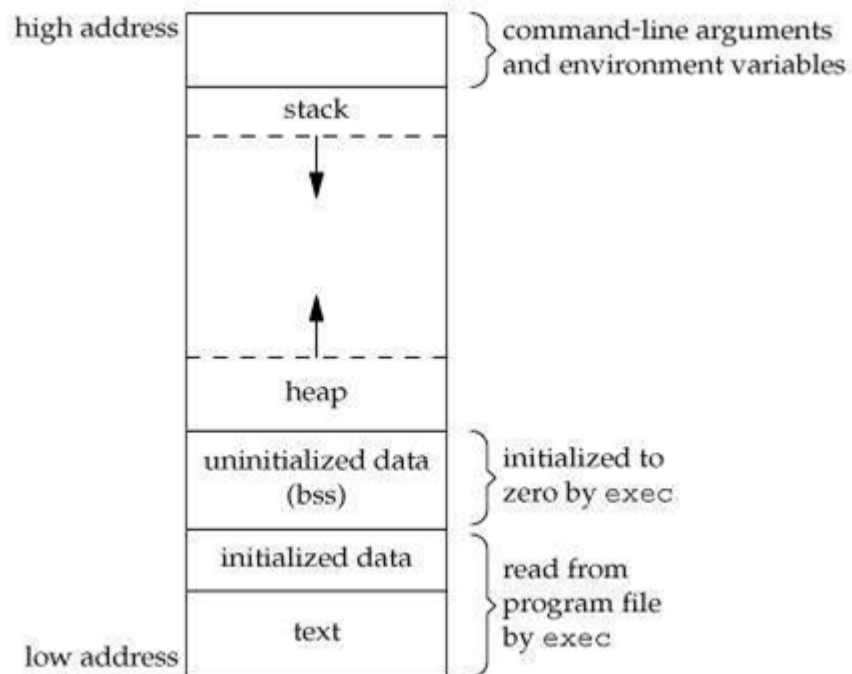
C++ 1차 레포트

1 프로그램 메모리 영역 (Code / Data / Stack / Heap)

Code 영역 (Text Segment)



SCALER
Topics



- 실행할 기계어 코드 저장
- 읽기 전용(Read-only) → 보안성 확보
- 프로그램 실행 중 변하지 않음

Data 영역

- 전역 변수, static 변수 저장
- 두 종류:
 - **Initialized Data**: 초기값 있음
 - **BSS**: 초기값 없음 (0으로 초기화)

Stack 영역

- 함수 호출 시 생성되는 지역 변수, 매개변수 저장
- 특징:
 - LIFO 구조
 - 자동 메모리 관리 (컴파일러)
 - 속도 빠름

- 크기 제한 있음

Heap 영역

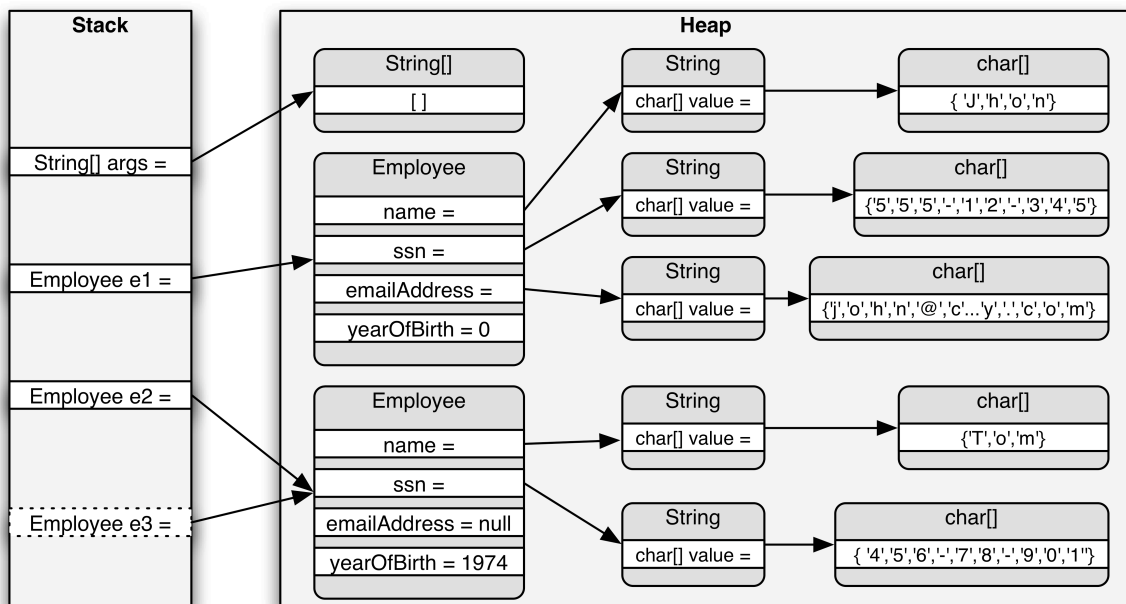
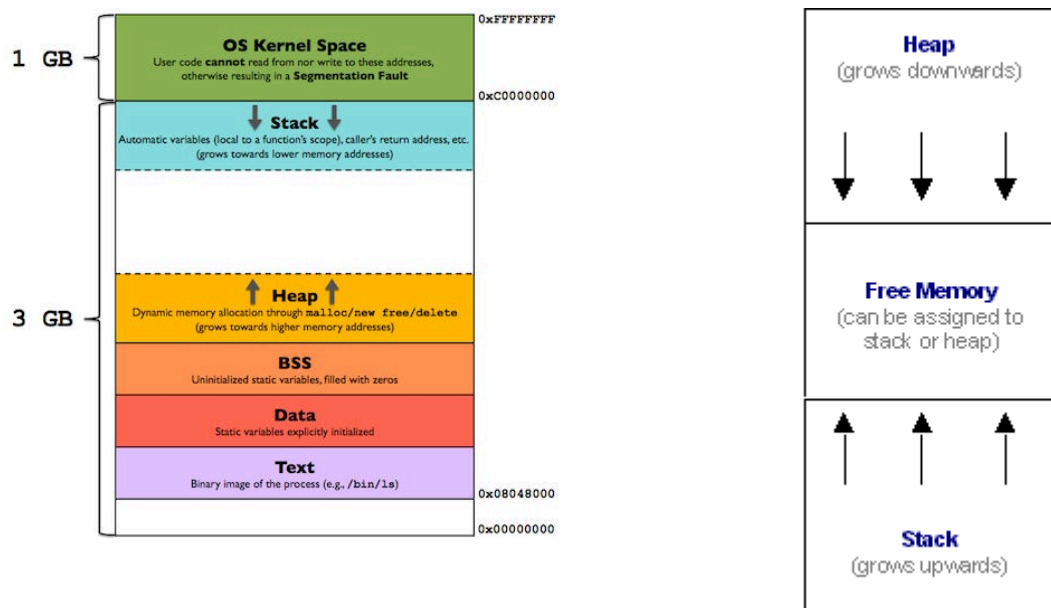
- 동적 메모리 할당 (malloc, new)
- 특징
 - 프로그래머가 직접 관리
 - 크기 유동적
 - 속도 느림
 - 메모리 누수 발생 가능

2 메모리 영역별 특성 비교

구분	Code	Data	Stack	Heap
저장 내용	프로그램 코드	전역/정적 변수	지역 변수	동적 할당
생성 시점	컴파일	프로그램 시작	함수 호출	실행 중
관리 방식	고정	고정	자동	수동
속도	빠름	빠름	매우 빠름	느림
크기	고정	고정	제한 있음	유동적

3 Stack과 Heap이 반대 방향으로 확장되는 이유

구조



이유

메모리 공간을 최대한 효율적으로 사용하기 위해

설명

- Stack: 높은 주소 → 낮은 주소 방향

- Heap: 낮은 주소 → 높은 주소 방향

👉 서로 반대로 확장하면:

1. 같은 공간 공유 가능
 2. 필요한 만큼 동적으로 확장
 3. 충돌 시 overflow 감지 가능
-

4 메모리 단편화 (Fragmentation)

1) 내부 단편화 (Internal)

- 할당된 블록 내부에 남는 공간 발생

예시: 10바이트 필요 → 16바이트 할당 → 6바이트 낭비

2) 외부 단편화 (External)

- 사용 가능한 메모리가 있지만

👉 연속되지 않아 할당 불가

해결 방법

Paging

- 메모리를 고정 크기 페이지로 나눔
 - 외부 단편화 해결
 - 대신 내부 단편화 발생
-

Segmentation

- 논리적 단위(코드, 데이터 등)로 분할
 - 가독성 좋음
 - 외부 단편화 발생 가능
-

+) 현대 방식

- Paging + Segmentation 혼합
- 가상 메모리 사용

5 메모리 누수 (Memory Leak)

정의

할당된 메모리를 해제하지 않아
사용 불가능한 상태로 남는 현상

예시 (C)

```
int *p = malloc(sizeof(int));  
// free(p); ← 없으면 누수 발생
```

해결 방법 비교

1 Garbage Collection (GC)

- 자동 메모리 관리
- 사용되지 않는 객체 자동 제거

특징

- 장점: 안전
- 단점: 성능 오버헤드

사용 언어

- Java, Python

2 Smart Pointer (C++)

```
std::unique_ptr<int> p = std::make_unique<int>(10);
```

특징

- 자동 해제 (RAII)
- GC보다 가볍고 빠름

비교

항목	GC	Smart Pointer
관리 방식	자동	반자동
성능	느림	빠름
안전성	높음	높음
사용 언어	Java, Python	C++

핵심 요약

- 프로그램 메모리는 Code, Data, Stack, Heap으로 구성된다.
- Stack은 함수 호출을 위한 자동 메모리 영역이며, Heap은 동적 할당을 위한 영역이다.
- Stack과 Heap은 메모리 효율성을 위해 서로 반대 방향으로 확장된다.
- 메모리 단편화는 내부/외부 단편화로 나뉘며, Paging과 Segmentation으로 해결할 수 있다.
- 메모리 누수는 해제되지 않은 메모리로 인해 발생하며, GC 또는 Smart Pointer로 해결할 수 있다.

추가 조사

- Paging은 메모리를 고정 크기 페이지와 프레임으로 나누어 관리하므로, 연속된 큰 공간을 찾을 필요가 없어 외부 단편화를 줄일 수 있다. 그러나 프로세스 크기가 페이지 크기의 배수가 아닌 경우 마지막 페이지 내부에 사용되지 않는 공간이 남아 내부 단편화가 발생한다.
- Segmentation은 코드, 데이터, 스택처럼 프로그램의 논리적 단위에 따라 가변 크기 세그먼트로 나누어 관리하므로 구조를 이해하고 보호하기 쉽다. 하지만 세그먼트 크기가 서로 달라 물리 메모리 내에 연속된 빈 공간을 확보해야 하므로 외부 단편화가 발생할 수 있다.